CSE 3204
Formal Language, Automata and Computability

Lecture 9
Minimization of DFA

Testing Equivalence of States

- Two states of a DFA p and q are equivalent if:

  - For all input strings w, $\hat{\delta}(p, w)$ is an accepting state if and only if $\hat{\delta}(q, w)$ is an accepting state.

If two states are not equivalent, then we say they are *distinguishable*. That is, state p is distinguishable from state q if there is at least one string w such that one of $\hat{\delta}(p, w)$ and $\hat{\delta}(q, w)$ is accepting, and the other is not accepting.

Example:

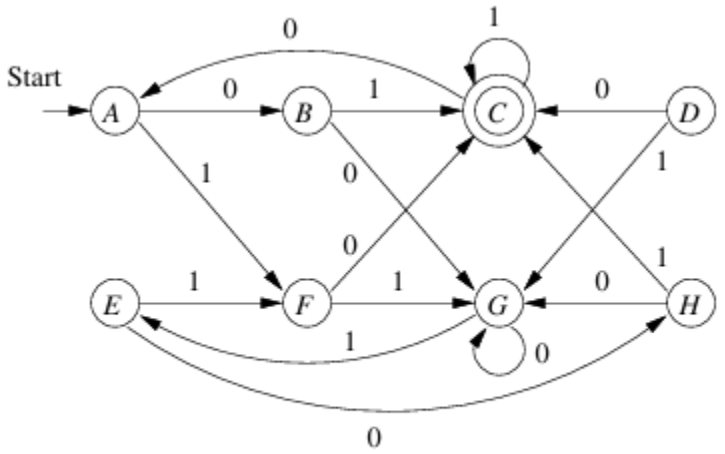

Figure 4.8: An automaton with equivalent states

State A and G:
∈ -- A, G
0 -- B, G
1 – F, E
01 – C, E –→ distinguishes A and G

State A and E:
∈ -- A, E, 0 – B, H, 1 – F, F 1x – leads to same state
0x – leads to same state States A and E are equivalent

Table Filling Algorithm

- A recursive discovery of distinguishable pairs in a DFA $A = (Q, \Sigma, \delta, q_0, F)$.

BASIS: If p is an accepting state and q is nonaccepting, then the pair $\{p, q\}$ is distinguishable.

INDUCTION: Let p and q be states such that for some input symbol a, $r = \delta(p, a)$ and $s = \delta(q, a)$ are a pair of states known to be distinguishable. Then $\{p, q\}$ is a pair of distinguishable states. The reason this rule makes sense is that there must be some string w that distinguishes r from s; that is, exactly one of $\hat{\delta}(r, w)$ and $\hat{\delta}(s, w)$ is accepting. Then string aw must distinguish p from q, since $\hat{\delta}(p, aw)$ and $\hat{\delta}(q, aw)$ is the same pair of states as $\hat{\delta}(r, w)$ and $\hat{\delta}(s, w)$.

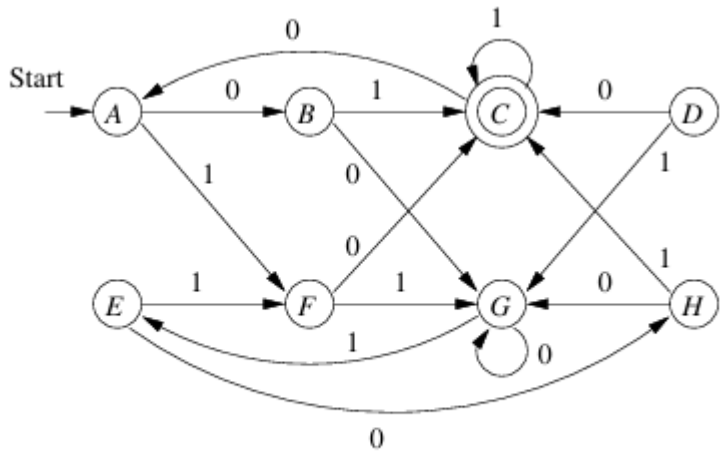

Figure 4.8: An automaton with equivalent states

	A	B	C	D	E	F	G
B	x						
C	x	x					
D	x	x	x				
E		x	x	x			
F	x	x	x		x		
G	x	x	x	x	x	x	
H	x		x	x	x	x	x

Figure 4.9: Table of state inequivalences

Equivalent states: {A, E}, {B, H} and {D, F}

States cannot be determined in the 1st pass: {A, G} and {E, G}

Table Filling Algorithm

Theorem 4.20: If two states are not distinguished by the table-filling algorithm, then the states are equivalent.

PROOF: Let us again assume we are talking of the DFA $A = (Q, \Sigma, \delta, q_0, F)$. Suppose the theorem is false; that is, there is at least one pair of states $\{p, q\}$ such that

1. States p and q are distinguishable, in the sense that there is some string w such that exactly one of $\hat{\delta}(p, w)$ and $\hat{\delta}(q, w)$ is accepting, and yet
2. The table-filling algorithm does not find p and q to be distinguished.

Call such a pair of states a *bad pair*.

If there are bad pairs, then there must be some that are distinguished by the shortest strings among all those strings that distinguish bad pairs. Let $\{p, q\}$ be one such bad pair, and let $w = a_1 a_2 \cdots a_n$ be a string as short as any that distinguishes p from q. Then exactly one of $\hat{\delta}(p, w)$ and $\hat{\delta}(q, w)$ is accepting.

Observe first that w cannot be ϵ, since if ϵ distinguishes a pair of states, then that pair is marked by the basis part of the table-filling algorithm. Thus, $n \geq 1$.

Consider the states $r = \delta(p, a_1)$ and $s = \delta(q, a_1)$. States r and s are distinguished by the string $a_2 a_3 \cdots a_n$, since this string takes r and s to the states $\hat{\delta}(p, w)$ and $\hat{\delta}(q, w)$. However, the string distinguishing r from s is shorter than any string that distinguishes a bad pair. Thus, $\{r, s\}$ cannot be a bad pair. Rather, the table-filling algorithm must have discovered that they are distinguishable.

But the inductive part of the table-filling algorithm will not stop until it has also inferred that p and q are distinguishable, since it finds that $\delta(p, a_1) = r$ is distinguishable from $\delta(q, a_1) = s$. We have contradicted our assumption that bad pairs exist. If there are no bad pairs, then every pair of distinguishable states is distinguished by the table-filling algorithm, and the theorem is true. □

Time Complexity of the Table Filling Algorithm

- n = number of states
- n (n-1)/2 = number of pairs of states
- $O(n^2)$ = time to execute one round
- $O(n^2)$ = number of rounds
- $O(n^4)$ = upper bound on the execution time of table filling algorithm.

- for each pair of states {r, s} find a list of {p, q} that depends on {r, s}
- For each {p, q} and for each symbol a, put {p, q} in the list of { δ (p, a), δ (q, a)}

If we ever find $\{r, s\}$ to be distinguishable, then we go down the list for $\{r, s\}$. For each pair on that list that is not already distinguishable, we make that pair distinguishable, and we put the pair on a queue of pairs whose lists we must check similarly.

- Since the size of the input alphabet is considered a constant each pair of states is put on O (1) lists. Also there are $O(n^2)$ pairs. The total work is $O(n^2)$.

Testing Equivalence of Regular Languages

Now, test if the start states of the two original DFA's are equivalent, using the table-filling algorithm. If they are equivalent, then $L = M$, and if not, then $L \neq M$.

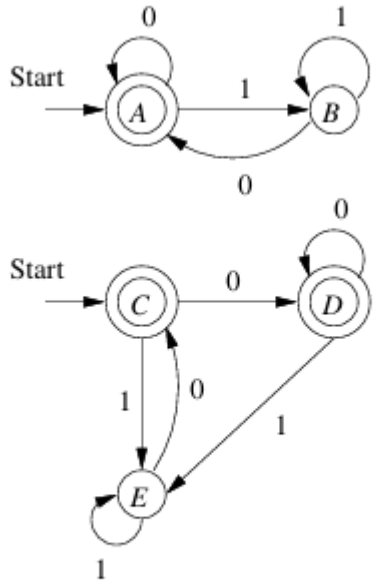

Figure 4.10: Two equivalent DFA's

Example 4.21 : Consider the two DFA's in Fig. 4.10. Each DFA accepts the empty string and all strings that end in 0; that is the language of regular expression $\epsilon + (\mathbf{0} + \mathbf{1})^*\mathbf{0}$. We can imagine that Fig. 4.10 represents a single DFA, with five states A through E. If we apply the table-filling algorithm to that automaton, the result is as shown in Fig. 4.11.

	A	B	C	D
B	x			
C		x		
D		x		
E	x		x	x

Figure 4.11: The table of distinguishabilities for Fig. 4.10

Equivalent pairs {A, C}, {A, D}, {B, E}, {C,D}

Minimization of DFA

1. First, eliminate any state that cannot be reached from the start state.

2. Then, partition the remaining states into blocks, so that all states in the same block are equivalent, and no pair of states from different blocks are equivalent. Theorem 4.24, below, shows that we can always make such a partition.

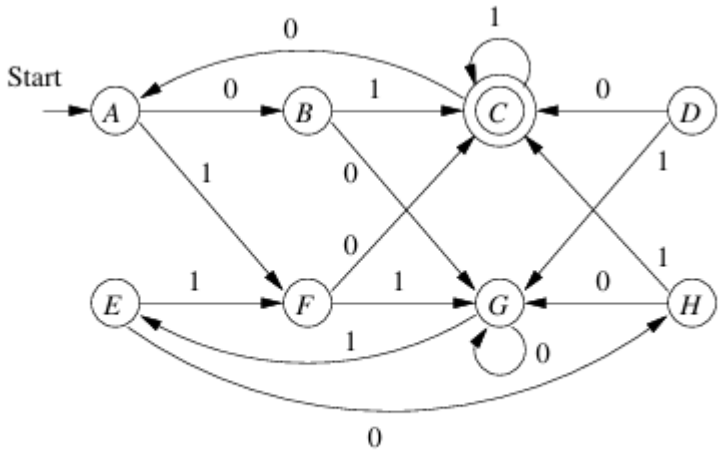

Figure 4.8: An automaton with equivalent states

	A	B	C	D	E	F	G
B	x						
C	x	x					
D	x	x	x				
E		x	x	x			
F	x	x	x		x		
G	x	x	x	x	x	x	
H	x		x	x	x	x	x

Figure 4.9: Table of state inequivalences

Equivalent states are
{A,E}, {B,H}, {F,D}
The partition of the states into blocks:
{A,E}, {B,H}, {C}, {F,D}, {G}

Minimization of DFA

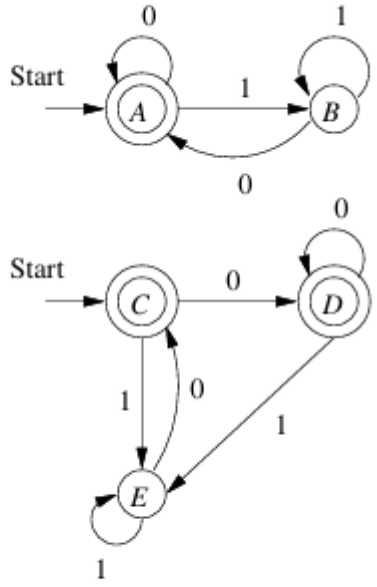

Figure 4.10: Two equivalent DFA's

	A	B	C	D
B	x			
C		x		
D		x		
E	x		x	x

Equivalent states are {A, C}, {A, D}, {B, E}, {C, D}
Partitions of states in blocks:
{A, C, D}, {B, E}

Theorem 4.23: The equivalence of states is transitive. That is, if in some DFA $A = (Q, \Sigma, \delta, q_0, F)$ we find that states p and q are equivalent, and we also find that q and r are equivalent, then it must be that p and r are equivalent.

Prove it by yourself.

Minimization of DFA

Theorem 4.24: If we create for each state q of a DFA a *block* consisting of q and all the states equivalent to q, then the different blocks of states form a *partition* of the set of states.[5] That is, each state is in exactly one block. All members of a block are equivalent, and no pair of states chosen from different blocks are equivalent. □

Minimization of DFA

We are now able to state succinctly the algorithm for minimizing a DFA $A = (Q, \Sigma, \delta, q_0, F)$.

1. Use the table-filling algorithm to find all the pairs of equivalent states.

2. Partition the set of states Q into blocks of mutually equivalent states by the method described above.

3. Construct the minimum-state equivalent DFA B by using the blocks as its states. Let γ be the transition function of B. Suppose S is a set of equivalent states of A, and a is an input symbol. Then there must exist one block T of states such that for all states q in S, $\delta(q, a)$ is a member of block T. For if not, then input symbol a takes two states p and q of S to states

   (a) The start state of B is the block containing the start state of A.

   (b) The set of accepting states of B is the set of blocks containing accepting states of A. Note that if one state of a block is accepting, then all the states of that block must be accepting. The reason is that any accepting state is distinguishable from any nonaccepting state, so you can't have both accepting and nonaccepting states in one block of equivalent states.

Minimization of DFA

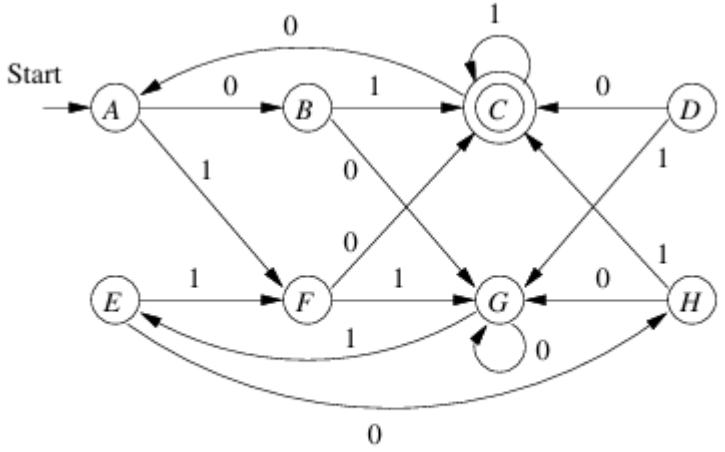

Figure 4.8: An automaton with equivalent states

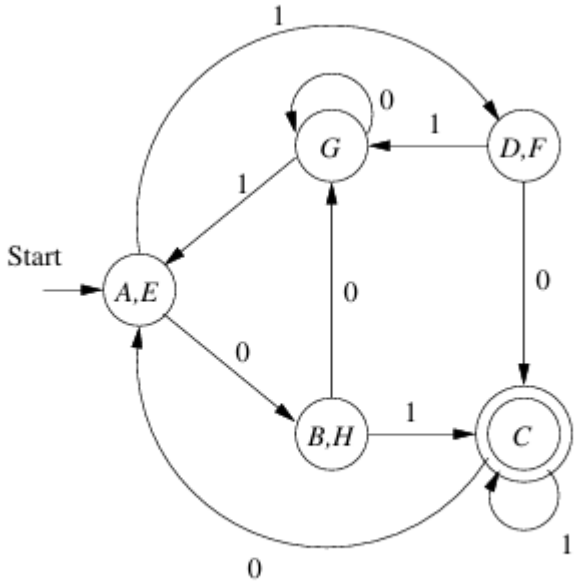

Figure 4.12: Minimum-state DFA equivalent to Fig. 4.8

Why the Minimized DFA Can't Be Beaten

- Suppose A is a DFA.
- M is minimized DFA from A using the partitioning method.
- Suppose N is another DFA unrelated to A that accepts the same language as A and M, yet has fewer states than M.
- Run the state distinguishability process on the states of M and N together.
- the transition function of the combined DFA is the union of the transition rules of M and N.
- States are accepting in the combined DFA if and only if they are accepting in the DFA from which they come.

Why the Minimized DFA Can't Be Beaten

- The start states of M and N are indistinguishable because L(M) = L(N)
- if {p,q} are indistinguishable then their successors on any one input symbol are also indistinguishable.
- Neither M nor N could have an inaccessible state or else we could eliminate that state and have an even smaller DFA for the same language.
- Every state of M is indistinguishable from at least one state of N.

Why the Minimized DFA Can't Be Beaten

p is a state of M. Then there is some string $a_1 a_2 \cdots a_k$ that takes the start state of M to state p. This string also takes the start state of N to some state q. Since we know the start states are indistinguishable, we also know that their

successors under input symbol a_1 are indistinguishable. Then, the successors of those states on input a_2 are indistinguishable, and so on, until we conclude that p and q are indistinguishable.

Since N has fewer states than M, there are two states of M that are indistinguishable from the same state of N, and therefore indistinguishable from each other. But M was designed so that all its states *are* distinguishable from each other. We have a contradiction, so the assumption that N exists is wrong.

Theorem 4.26: If A is a DFA, and M the DFA constructed from A by the algorithm described in the statement of Theorem 4.24, then M has as few states as any DFA equivalent to A. □

- Section 4.4 of Chapter 4